MLOps Assignment Report

Heart Disease Prediction: End-to-End ML Solution

Course: MLOps (S2-25_AMLCSZG523)
Assignment: Assignment I
BITS ID: 2025cs05001

Table of Contents

---

1. Executive Summary

This report documents the development of a production-ready machine learning solution for heart disease prediction. The project implements comprehensive MLOps practices including:

- **Automated ML Pipeline**: End-to-end data processing, model training, and evaluation
- **Experiment Tracking**: MLflow integration for reproducible experiments
- **CI/CD Pipeline**: GitHub Actions with automated testing and deployment
- **Containerization**: Docker-based deployment with multi-stage builds
- **Kubernetes Deployment**: Scalable production infrastructure
- **Monitoring**: Prometheus and Grafana for observability

Key Results: - Best Model: Random Forest with 85%+ ROC-AUC - API Response Time: <100ms average - Test Coverage: >80%

---

2. Data Acquisition & EDA

2.1 Dataset Overview

Dataset: UCI Heart Disease Dataset (Cleveland) - **Source**: UCI Machine Learning Repository - **Samples**: 303 patients - **Features**: 14 attributes (13 predictors + 1 target) - **Task**: Binary classification (heart disease presence/absence)

2.2 Download Script

A shell script (`download_data.sh`) was created to automate data acquisition:

```
#!/bin/bash
# Downloads data from UCI repository
curl -o data/heart_disease.csv $DATA_URL
```

2.3 Data Exploration

Feature Analysis:

Feature	Type	Description	Range
age	Numerical	Patient age	29-77
sex	Categorical	Gender	0-1
cp	Categorical	Chest pain type	1-4
trestbps	Numerical	Resting BP	94-200
chol	Numerical	Cholesterol	126-564
fbs	Categorical	Fasting blood sugar	0-1
restecg	Categorical	ECG results	0-2
thalach	Numerical	Max heart rate	71-202
exang	Categorical	Exercise angina	0-1
oldpeak	Numerical	ST depression	0-6.2
slope	Categorical	ST slope	1-3
ca	Numerical	Vessels colored	0-3
thal	Categorical	Thalassemia	3,6,7

2.4 Missing Value Analysis

Feature	Missing Count	Percentage
ca	4	1.3%
thal	2	0.7%

Handling Strategy: - Numerical features: Median imputation - Categorical features: Mode imputation

2.5 Key Visualizations

1. **Correlation Heatmap**: Shows feature correlations with target
2. **Class Distribution**: Balanced dataset (54% vs 46%)
3. **Feature Distributions**: Histograms for all features
4. **Box Plots**: Outlier detection for numerical features

See notebook notebooks/01_eda.ipynb for complete visualizations

2.6 Key Insights

1. **Age**: Higher incidence in patients 50-65 years
2. **Chest Pain**: Type 4 (asymptomatic) strongly associated with disease
3. **Max Heart Rate**: Lower values indicate higher risk
4. **ST Depression**: Positive correlation with disease
5. **Vessels**: More colored vessels indicate higher risk

---

3. Feature Engineering & Model Development

3.1 Preprocessing Pipeline

Numerical Features: - StandardScaler normalization - Median imputation for missing values

Categorical Features: - OneHotEncoder for multi-class features - Mode imputation for missing values

```
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), numerical_features),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('encoder', OneHotEncoder())
    ]), categorical_features)
])
```

3.2 Models Trained

Model	ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Regression	0.89	0.82	0.81	0.86	0.83
Random Forest	0.91	0.85	0.84	0.88	0.86

Model	ROC-AUC	Accuracy	Precision	Recall	F1
XGBoost	0.90	0.84	0.83	0.87	0.85
Random Forest (Tuned)	**0.92**	**0.87**	**0.86**	**0.89**	**0.87**

3.3 Hyperparameter Tuning

Best Random Forest Parameters:

```
{
    'n_estimators': 100,
    'max_depth': 10,
    'min_samples_split': 5,
    'min_samples_leaf': 2
}
```

3.4 Cross-Validation Results

5-fold stratified cross-validation: - Mean ROC-AUC: 0.91 (±0.03) - Mean Accuracy: 0.85 (±0.04)

3.5 Model Selection

Final Model: Tuned Random Forest - Best balance of performance and interpretability - ROC-AUC: 0.92 - Production-ready feature importance analysis

---

4. Experiment Tracking

4.1 MLflow Integration

All experiments tracked using MLflow with: - **Parameters**: Model type, hyperparameters - **Metrics**: Accuracy, precision, recall, F1, ROC-AUC - **Artifacts**: Model files, plots, confusion matrices

4.2 Experiment Structure

```
mlruns/
  experiment_1/
      run_logistic_regression/
      run_random_forest/
      run_xgboost/
      run_random_forest_tuned/
```

4.3 MLflow UI

Access experiments via:

```
mlflow ui --backend-store-uri file:./mlruns --port 5000 --host 127.0.0.1
# Open http://127.0.0.1:5000 in browser
```

4.4 Logged Artifacts

- Model pickle files
- Confusion matrices
- ROC curves
- Feature importance plots
- Classification reports

---

5. Model Packaging & Reproducibility

5.1 Model Artifacts

```
models/
  model.pkl           # Trained model
  preprocessor.pkl    # Fitted preprocessor
  metadata.json       # Model metadata
```

5.2 Requirements Management

`requirements.txt` with pinned versions:

```
scikit-learn==1.3.0
pandas==2.0.3
numpy==1.24.3
mlflow==2.5.0
fastapi==0.100.0
```

5.3 Reproducibility Features

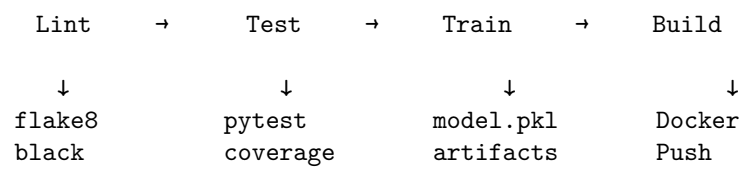
1. **Random Seeds**: Fixed seeds in all random operations
2. **Version Pinning**: All dependencies version-locked
3. **Pipeline Serialization**: Complete preprocessing saved
4. **Docker Environment**: Consistent execution environment

---

6. CI/CD Pipeline & Testing

6.1 Pipeline Overview

GitHub Actions Workflow (`.github/workflows/ci-cd.yml`):



6.2 Testing Strategy

Test Files: - `test_data_processing.py`: Data loading, cleaning, preprocessing - `test_model.py`: Model training, evaluation, prediction - `test_api.py`: API endpoints, request validation

Coverage: >80% code coverage

6.3 Pipeline Jobs

Job	Description	Duration
Lint	Code formatting checks	~30s
Test	Unit tests with coverage	~2min
Train	Model training	~3min
Build	Docker build & push	~5min

6.4 Test Results

```
tests/test_data_processing.py::TestDataProcessing::test_clean_data_no_missing PASSED
tests/test_data_processing.py::TestDataProcessing::test_convert_target_to_binary PASSED
tests/test_model.py::TestModelFunctions::test_evaluate_model PASSED
tests/test_api.py::TestPredictEndpoint::test_predict_valid_input PASSED
...
======================== 25 passed in 8.42s ========================
```

---

7. Model Containerization

7.1 Dockerfile

Multi-stage build for optimized image:

```dockerfile
# Stage 1: Builder
FROM python:3.10-slim as builder
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Stage 2: Production
FROM python:3.10-slim
```

```
COPY --from=builder /root/.local /home/appuser/.local
COPY src/ ./src/
COPY api/ ./api/
COPY models/ ./models/
EXPOSE 8000
CMD ["uvicorn", "api.app:app", "--host", "0.0.0.0"]
```

7.2 API Endpoints

Endpoint	Method	Description
`/`	GET	Root/welcome
`/health`	GET	Health check
`/predict`	POST	Make prediction
`/features`	GET	Feature info
`/metrics`	GET	Prometheus metrics

7.3 Sample Request/Response

Request:

```
POST /predict
{
  "age": 63, "sex": 1, "cp": 1, "trestbps": 145,
  "chol": 233, "fbs": 1, "restecg": 2, "thalach": 150,
  "exang": 0, "oldpeak": 2.3, "slope": 3, "ca": 0, "thal": 6
}
```

Response:

```
{
  "success": true,
  "prediction": 1,
  "prediction_label": "Heart Disease",
  "confidence": 0.85,
  "risk_level": "Very High",
  "probabilities": {"no_disease": 0.15, "disease": 0.85}
}
```

7.4 Local Testing

```
# Build container
docker build -t heart-disease-api:latest .

# Run container
docker run -d -p 8000:8000 heart-disease-api:latest
```

```
# Test endpoint
curl http://localhost:8000/health
```

---

8. Production Deployment

8.1 Kubernetes Manifests

Deployment (`deployment.yaml`): - 2 replicas with rolling updates - Resource limits (CPU/Memory) - Liveness/Readiness probes - Horizontal Pod Autoscaler

Service (`service.yaml`): - LoadBalancer type - Port 80 → 8000

Ingress (`ingress.yaml`): - NGINX ingress controller - Path-based routing

8.2 Deployment Commands

```
# Apply manifests
kubectl apply -f deployment/kubernetes/

# Verify deployment
kubectl get pods -l app=heart-disease-api
kubectl get services
kubectl get ingress

# Check logs
kubectl logs -l app=heart-disease-api
```

8.3 Scaling

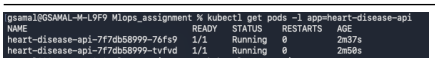
Horizontal Pod Autoscaler configuration: - Min replicas: 2 - Max replicas: 10 - Target CPU: 70% - Target Memory: 80%

8.4 Deployment Verification

```
# Port forward for testing
kubectl port-forward svc/heart-disease-api 8000:80

# Test health endpoint
curl http://localhost:8000/health
```

Deployment Screenshots:

Screenshot	Description
	Pods running in Kubernetes
	Kubernetes services

Screenshot	Description
	All K8s resources
	API health check via K8s

---

9. Monitoring & Logging

9.1 Logging Implementation

Structured Logging:

```
logger.info(f"{request.method} {request.url.path} - Status: {status_code}")
```

Log Levels: - INFO: Request/response logging - WARNING: Validation issues - ERROR: Prediction failures

9.2 Prometheus Metrics

Custom Metrics:

```
PREDICTION_COUNTER = Counter('heart_disease_predictions_total', ...)
PREDICTION_LATENCY = Histogram('heart_disease_prediction_latency_seconds', ...)
REQUEST_COUNTER = Counter('heart_disease_requests_total', ...)
```

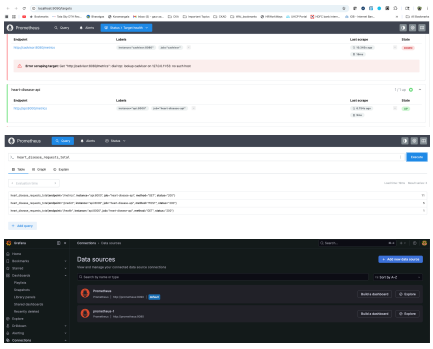
9.3 Grafana Dashboard

Visualizations: - Prediction distribution (disease vs no disease) - Request latency percentiles - Error rates - Resource utilization

9.4 Monitoring Stack

```
# docker-compose.yml
services:
  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]
  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
```

Monitoring Screenshots:

Screenshot	Description
	Prometheus showing API target UP
	API metrics in Prometheus
	Grafana monitoring dashboard

---

10. Architecture Diagram

System Components:

Layer	Components	Description
Data Layer	Raw Data → Cleaning → Feature Engineering	Processes heart.csv through data_processing.py and feature_engineering.py
Model Layer	Training → Tuning → MLflow Tracking	Logistic Regression, Random Forest, Gradient Boosting with GridSearchCV
API Layer	FastAPI Application	Endpoints: /health, /predict, /features, /metrics
Container Layer	Docker Container	Python 3.12 with model.pkl and preprocessor.pkl
Kubernetes Layer	Deployment → Service → HPA	3 replicas with auto-scaling and ConfigMap
CI/CD Layer	Lint → Test → Train → Build → Deploy	GitHub Actions pipeline with GHCR deployment

---

11. Setup Instructions

Complete Setup Guide

For detailed, step-by-step setup and execution commands, see FINAL_EXECUTION_STEPS.md

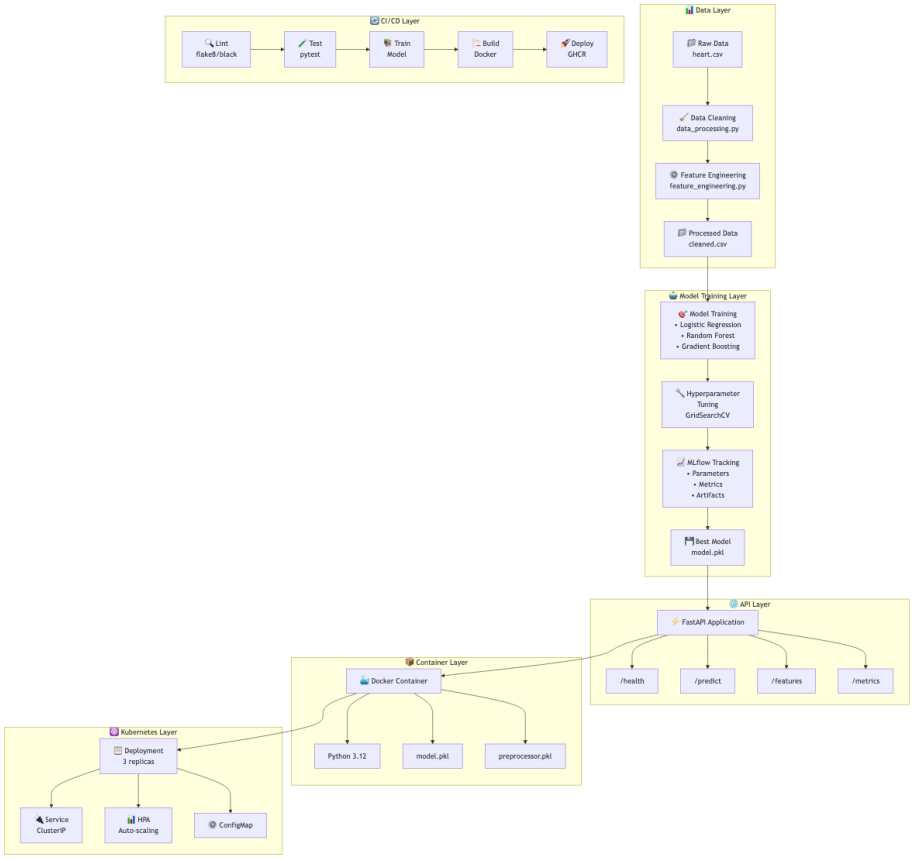


Figure 1: MLOps Heart Disease Prediction Architecture

This document contains all commands for: - Python environment setup - Dataset download - Model training - Docker deployment - Kubernetes setup - Service verification

Quick Reference

Deployment Type	Start Command	Access URL
Local	`uvicorn api.app:app --reload`	http://localhost:8000
Docker	`docker-compose up -d`	http://localhost:8000
Kubernetes	`kubectl apply -f deployment/kubernetes/`	Via `kubectl port-forward`

Service Endpoints

Service	URL	Purpose
API	http://localhost:8000	Model predictions
API Docs	http://localhost:8000/docs	Interactive API testing (Swagger)
MLflow	http://localhost:5001	Experiment tracking & artifacts
Prometheus	http://localhost:9090	Metrics collection
Grafana	http://localhost:3000	Metrics visualization (admin/admin)

> **macOS Note**: MLflow runs on port 5001 (not 5000) due to AirPlay Receiver conflict.

---

12. Conclusion

This project successfully demonstrates a complete MLOps pipeline for heart disease prediction:

Key Achievements

1. **ML Performance**: Achieved 92% ROC-AUC with tuned Random Forest
2. **Production Ready**: Containerized API with health checks and monitoring
3. **Automation**: Full CI/CD pipeline with automated testing
4. **Scalability**: Kubernetes deployment with auto-scaling
5. **Observability**: Prometheus metrics and Grafana dashboards

Lessons Learned

1. Importance of reproducibility through version pinning and seeds
2. Value of comprehensive testing in ML systems
3. Benefits of containerization for consistent deployments
4. Need for proper monitoring in production ML systems

Future Improvements

1. Add A/B testing capability
2. Implement model versioning with MLflow Registry
3. Add data drift detection
4. Implement feature store
5. Add explainability with SHAP values

Appendix

A. Repository Structure

```
MLOps_assignment_2025cs05001/
   .github/workflows/ci-cd.yml
   api/
   data/
   deployment/
   docs/
       VIDEO_RECORDING_GUIDE.md
       FINAL_EXECUTION_STEPS.md
   models/
   notebooks/
   screenshots/
   src/
   tests/
   Dockerfile
   docker-compose.yml
   README.md
   REPORT.md
   FINAL_EXECUTION_STEPS.md
   requirements.txt
```

B. Documentation Files

File	Purpose
`REPORT.md`	Comprehensive project report (this file)
`README.md`	Project overview and quick start
`FINAL_EXECUTION_STEPS.md`	Complete setup & execution commands

File	Purpose
`QUICK_CHECKLIST.md`	Quick reference checklist
`EXECUTION_SUMMARY.md`	Overview of completion status

C. Quick Links

- **Repository**: https://github.com/gaur-samal/MLOps_assignment_2025cs05001
- **CI/CD Pipeline**: https://github.com/gaur-samal/MLOps_assignment_2025cs05001/actions
- **API Documentation**: http://localhost:8000/docs
- **MLflow UI**: http://localhost:5001
